

Clúster Kind con Soporte para GPU

Documentación Completa

13 de noviembre de 2025

Resumen

Esta documentación proporciona una guía completa y profesional para configurar un clúster Kubernetes local utilizando Kind con soporte avanzado para GPUs NVIDIA. Incluye instalación automatizada, scripts, ejemplos prácticos y soluciones a problemas comunes, facilitando el aprendizaje y desarrollo en entornos de inteligencia artificial y computación intensiva. Dirigido a desarrolladores, investigadores y estudiantes, el documento democratiza el acceso a tecnologías avanzadas sin depender de recursos costosos en la nube.

1 Prólogo

En un mundo donde la computación en la nube y la inteligencia artificial avanzan a pasos agigantados, la necesidad de entornos de desarrollo locales eficientes y accesibles se vuelve cada vez más crítica. Esta documentación surge de la experiencia práctica en el despliegue de clústeres de Kubernetes utilizando Kind, con un enfoque especial en la integración de GPU para aplicaciones de alto rendimiento.

Como Líder de IA y Arquitecto de Soluciones, he observado cómo las barreras técnicas pueden frenar la innovación. Esta guía no solo proporciona instrucciones técnicas detalladas, sino que también busca democratizar el acceso a tecnologías avanzadas, permitiendo a desarrolladores, investigadores y estudiantes experimentar con Kubernetes y GPU sin depender de recursos en la nube costosos o complejos.

El objetivo es empoderar a la comunidad tecnológica, facilitando el aprendizaje y la experimentación. Cada sección ha sido diseñada pensando en la practicidad, con ejemplos reales, soluciones a problemas comunes y herramientas automatizadas que reducen la curva de aprendizaje.

Que esta documentación sea un puente entre la teoría y la práctica, inspirando nuevas ideas y proyectos en el fascinante mundo de la orquestación de contenedores y la computación acelerada.

William R. (wisrovi-rodriguez)

Índice

1	Prólogo	2
2	Acerca del Autor	10
3	Introducción	11
3.1	¿Qué es Kind?	11
3.2	Beneficios de Usar Kind con GPUs	11
3.3	Glosario	11
4	Instalación de Dependencias	13
4.1	Instalar Docker	13
4.2	Instalar NVIDIA Container Toolkit	14
4.3	Instalar Docker Compose	16
4.4	Instalar kubectl	16
4.5	Instalar containerd	17
4.6	Instalar etcd	18
4.7	Instalar kubelet	19
5	Objetivos del Proyecto	21
6	Requisitos del Sistema	22
6.1	Hardware	22
6.2	Software	22
6.3	Red	23
6.4	Permisos	23
6.5	Verificación de Prerrequisitos	23
7	Instalación de Herramientas	24
7.1	Instalar Kind	24
8	Arquitectura del Clúster	25
8.1	Componentes	25
8.1.1	Nodos	25
8.1.2	Redes	25

8.1.3	Acceso a GPU	25
8.1.4	Almacenamiento	26
8.2	Seguridad	26
8.3	Limitaciones	26
9	Creación y Configuración del Clúster	27
9.1	Creación del Clúster	27
9.2	Eliminación del Clúster	27
9.3	Verificación de Creación	27
9.4	Consideraciones Adicionales	27
9.5	Configuración de GPU	28
9.6	Montajes de Dispositivos	28
9.7	Programación de GPU en Kubernetes	28
10	Verificación y Pruebas	29
11	Pruebas	30
11.1	Validación de Acceso a GPU	30
11.2	Prueba Automatizada	31
12	Solución de Problemas	32
12.1	Creación del Clúster Falla	32
12.2	Nodos No Listos	32
12.3	GPU No Detectada	32
12.4	Problemas de Exposición de Puertos	32
12.5	Errores del Plugin de Dispositivo	32
12.6	Recrear Clúster	32
13	Limpieza y Mantenimiento	33
14	Preguntas Frecuentes	34
14.1	General	34
14.2	Configuración	34
14.3	GPU	34
14.4	Uso	34

14.5 Solucion de Problemas	35
15 Ejemplos de Uso	36
15.1 Ejecutar Pod GPU	36
15.2 Ejemplo de Despliegue de una Aplicación Simple	37
15.3 Ejemplo de Uso de GPU para Computación Científica	37
15.4 Ejemplo de Escalado de Pods	37
15.5 Ejemplo de Monitoreo con k9s	38
15.6 Ejemplo de Despliegue con ArgoCD	38
15.7 Ejemplo de Configuración de Recursos GPU Avanzada	38
15.8 Ejemplo de Debugging de Pods con GPU	39
15.9 Ejemplo de Backup y Restauración de Configuración	39
16 Herramientas Adicionales	40
16.0.1 Instalacion de ArgoCD	40
16.0.2 Instalacion de k9s	40
16.0.3 Integración con Lens	43
17 Registro de Cambios	45
17.1 [1.1.0] - 2025-11-13	45
17.1.1 Agregado	45
17.1.2 Características	45
17.2 [1.0.0] - 2025-11-12	45
17.2.1 Agregado	45
17.2.2 Características	45
18 Licencia	46
19 Apéndices	47
19.1 Versión de Componentes	47
19.2 Estructura del Proyecto	47
20 Anexos	49
20.1 16.1: Script setup.sh	49
20.2 16.2: Script validate.sh	50

20.3 16.3: Script cleanup.sh	50
20.4 16.4: Script monitoring.sh	51
20.5 16.5: Script set_path.sh	51
20.6 16.6: Archivo kind-config.yaml	52
20.7 16.8: Script mega_setup.sh	55
20.8 16.7: Archivo kubeconfig.yaml	62
20.9 16.9: Script cluster_manager.py	62
20.10 16.10: Script gpu_monitor.py	62
20.11 16.11: Archivo argocd-app.yaml	62
21 Bibliografía	64

Índice de figuras

1	Logo de Kind	11
2	Arquitectura de Docker	13
3	Salida del comando docker --version	14
4	Logo de NVIDIA	14
5	Logo de Docker Compose	16
6	Logo de kubectl	16
7	Salida del comando kubectl version --client	17
8	Salida del comando kubectl version --client	17
9	Logo de containerd	17
10	Salida del comando containerd --version	18
11	Logo de etcd	18
12	Salida del comando etcd --version	19
13	Salida del comando kubelet --version	19
14	Guía de inicio rápido de Kind	24
15	Salida del comando kind --version	24
16	Arquitectura de Kubernetes	25
17	Diagrama del clúster Kind	26
18	Operador GPU: Host a Kubernetes	28
19	Salida de nvidia-smi en worker	30
20	Diagrama de un Pod en Kubernetes	36
21	Logo de ArgoCD	40
22	Arquitectura de ArgoCD	41
23	Interfaz de usuario de ArgoCD	42
24	Logo de k9s	42
25	Captura de pantalla de k9s	43
26	Captura de pantalla de Lens	44

Índice de cuadros

1	Requisitos de Hardware	22
2	Requisitos de Software	22
3	Requisitos de Red	23
4	Requisitos de Permisos	23

Listings

1	Instalación de Docker en Ubuntu	13
2	Instalación del NVIDIA Container Toolkit	14
3	Salida del comando nvidia-smi	15
4	Instalación de Docker Compose	16
5	Instalación de kubectl	17
6	Instalación de containerd	17
7	Instalación de etcd	18
8	Instalación de kubelet	19
9	Verificación del estado del clúster	29
10	Validación de acceso a GPU en nodos trabajadores	30
11	Instalación de ArgoCD en el clúster	40
12	Verificación de pods de ArgoCD	40
13	Acceso a la interfaz web de ArgoCD	40
14	Descarga e instalación de k9s	41
15	Configuración temporal de PATH para k9s	42
16	Configuración permanente de PATH para k9s	42
17	Ejecución de k9s	42
18	Estructura del proyecto	47

2 Acerca del Autor

William R. (wisrovi-rodriguez)

Líder de IA y Arquitecto de Soluciones en eCaptureDtech.
Ubicación: Badajoz, Extremadura, España.



Como Líder de IA y Arquitecto de Soluciones en eCaptureDtech, mi misión es forjar el futuro de la tecnología mediante la integración de inteligencia artificial avanzada en soluciones empresariales. Con una sólida formación en ingeniería y una pasión por la innovación, he dedicado mi carrera a explorar las fronteras de la IA, desde el desarrollo de algoritmos de aprendizaje automático hasta la implementación de sistemas de IA generativa.

Mi experiencia abarca desde la investigación académica hasta el liderazgo en proyectos tecnológicos, colaborando con equipos multidisciplinares para resolver desafíos complejos. Soy un apasionado de la educación y el compartir conocimiento, contribuyendo activamente a la comunidad tecnológica a través de publicaciones, conferencias y proyectos open-source.

En mi rol actual, me enfoco en diseñar arquitecturas escalables que aprovechen el poder de la IA para transformar industrias, siempre con un enfoque ético y sostenible.

Contacto:

LinkedIn: <https://es.linkedin.com/in/wisrovi-rodriguez>

GitHub: <https://github.com/wisrovi>

3 Introducción

Esta documentación aborda la configuración de un clúster Kubernetes local usando Kind con soporte para GPUs NVIDIA, facilitando el desarrollo y aprendizaje en entornos de IA. Siguiendo la visión del prólogo de democratizar tecnologías avanzadas, el documento proporciona una guía paso a paso, desde prerequisites hasta ejemplos avanzados.

3.1 ¿Qué es Kind?

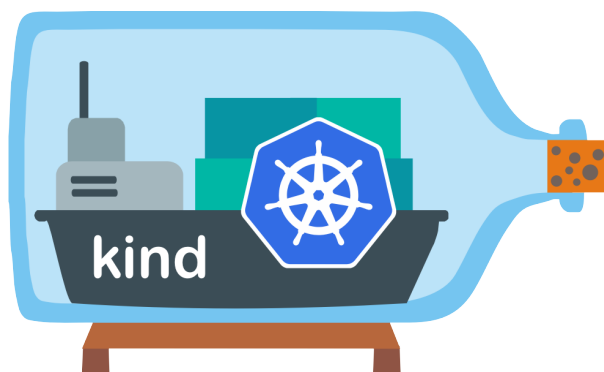


Figura 1: Logo de Kind

Kind (Kubernetes in Docker) es una herramienta que ejecuta clústeres Kubernetes locales usando contenedores Docker. Ideal para desarrollo, pruebas y aprendizaje, permite experimentar con configuraciones avanzadas sin costos de nube.

3.2 Beneficios de Usar Kind con GPUs

Integra GPUs para procesamiento local de IA, desarrollo eficiente, ahorro de costos y reproducibilidad en entornos locales.

3.3 Glosario

ArgoCD Herramienta de GitOps para Kubernetes que automatiza el despliegue y gestión de aplicaciones.

Cluster Conjunto de nodos interconectados que trabajan juntos para ejecutar aplicaciones distribuidas.

Docker Plataforma de contenedorización que permite empaquetar, distribuir y ejecutar aplicaciones en entornos aislados llamados contenedores.

GPU Unidad de Procesamiento Gráfico, utilizada para acelerar tareas de computación intensiva como el procesamiento de imágenes y aprendizaje automático.

k9s Interfaz de usuario basada en terminal para gestionar clústeres de Kubernetes de manera eficiente.

Kind Herramienta para ejecutar clústeres de Kubernetes locales utilizando contenedores Docker como nodos.

Kubernetes Plataforma de orquestación de contenedores de código abierto para automatizar el despliegue, escalado y gestión de aplicaciones en contenedores.

Node Máquina física o virtual en un clúster de Kubernetes que ejecuta pods y es gestionada por el plano de control.

NVIDIA Empresa líder en tecnología de GPUs y computación paralela, proveedora de drivers y toolkits para GPUs.

Pod La unidad más pequeña y básica en Kubernetes, que representa un conjunto de contenedores que comparten recursos y red.

4 Instalación de Dependencias

Antes de comenzar con Kind, es necesario instalar las herramientas básicas como Docker, NVIDIA Container Toolkit y Docker Compose.

4.1 Instalar Docker

Docker es una plataforma de contenedorización de código abierto que permite empaquetar, distribuir y ejecutar aplicaciones en entornos aislados llamados contenedores. Es esencial para Kind, ya que Kind utiliza contenedores Docker para simular nodos de Kubernetes, proporcionando un entorno ligero y reproducible.

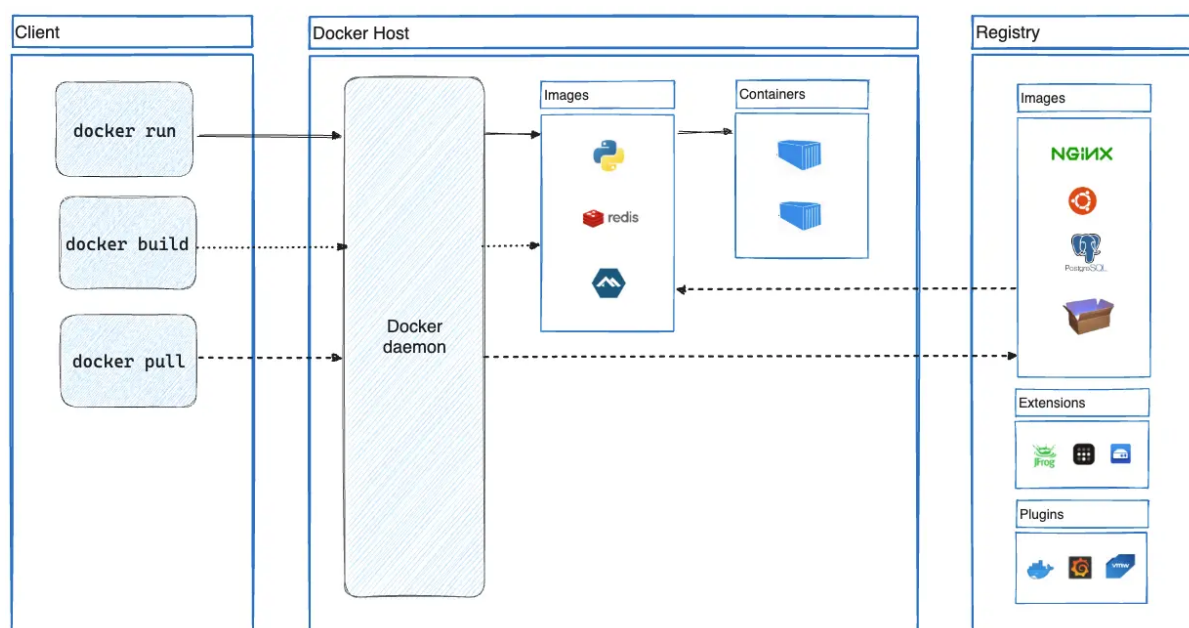


Figura 2: Arquitectura de Docker

Basado en: <https://docs.docker.com/engine/install/ubuntu/>

```

1 # Actualizar el sistema
2 sudo apt update
3
4 # Instalar Docker
5 sudo apt install -y docker.io
6
7 # Iniciar y habilitar Docker
8 sudo systemctl start docker
9 sudo systemctl enable docker
10
11 # Agregar usuario al grupo docker (opcional)
12 sudo usermod -aG docker $USER
13
14 # Verificar instalacion
15 docker --version

```

Listing 1: Instalación de Docker en Ubuntu

Salida esperada:

```
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
```

Figura 3: Salida del comando `docker --version`

4.2 Instalar NVIDIA Container Toolkit



Figura 4: Logo de NVIDIA

El NVIDIA Container Toolkit permite que los contenedores Docker accedan a GPUs NVIDIA, habilitando el paso a través de dispositivos para aplicaciones que requieren aceleración GPU. Es crucial para ejecutar cargas de trabajo de IA o computación intensiva en contenedores.

Basado en: <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>

```

1  # Instalar prerequisites
2  sudo apt-get update && sudo apt-get install -y --no-install-recommends \
3      curl \
4      gnupg2
5
6  # Configurar repositorio
7  curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | sudo
8      gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-
9      keyring.gpg \
10     && curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb
11     /nvidia-container-toolkit.list | \
12     sed 's#deb_https://#deb_[signed-by=/usr/share/keyrings/nvidia-
13     container-toolkit-keyring.gpg]\_https://#g' | \
14     sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
15
16 # Actualizar paquetes
17 sudo apt-get update
18
19 # Instalar NVIDIA Container Toolkit
20 export NVIDIA_CONTAINER_TOOLKIT_VERSION=1.18.0-1
21 sudo apt-get install -y \
22     nvidia-container-toolkit=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
23     nvidia-container-toolkit-base=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \
24     libnvidia-container-tools=${NVIDIA_CONTAINER_TOOLKIT_VERSION} \

```

```

21     libnvidia-container1=${NVIDIA_CONTAINER_TOOLKIT_VERSION}
22
23 # Configurar runtime para Docker
24 sudo nvidia-ctk runtime configure --runtime=docker
25 sudo systemctl restart docker
26
27 # Verificar
28 docker run --rm --gpus all nvidia/cuda:11.0-base nvidia-smi

```

Listing 2: Instalación del NVIDIA Container Toolkit

Salida esperada (ejemplo):

```

1 Thu Nov 13 08:58:20 2025
2 +-----+-----+-----+-----+-----+-----+
3 | NVIDIA-SMI 575.64.03                  Driver Version: 575.64.03          CUDA
4 | Version: 12.9                        |                               |
5 |-----+-----+-----+-----+-----+-----+
6 | GPU   Name                               Persistence-M | Bus-Id        Disp.A |
7 | Volatile Uncorr. ECC |                               |               |    |
8 | Fan  Temp  Perf    Pwr:Usage/Cap |                               |               |    |
9 | GPU-Util  Compute M. |                               |               |    |
10 |                               |                               |               |    |
11 | MIG M. |                               |               |    |
12 |=====+=====+=====+=====+=====+=====+
13 |    0   NVIDIA GeForce RTX 3060                Off |   00000000:01:00.0   On |
14 |                               N/A |                               |               |    |
15 |    0%    39C    P8              14W / 170W |   1861MiB / 12288MiB |
16 |    30%    Default |                               |               |    |
17 |                               |                               |               |    |
18 |                               N/A |                               |               |    |
19 |-----+-----+-----+-----+-----+-----+
20 | Processes:
21 |
22 | GPU   GI    CI          PID    Type   Process name
23 |                               GPU Memory |
24 |                               |
25 |                               ID    ID
26 |                               |
27 |                               |
28 |=====+=====+=====+=====+=====+=====+
29 |    0   N/A  N/A            5157     G   /usr/lib/xorg/Xorg
30 |                               357MiB |
31 ...

```

Listing 3: Salida del comando nvidia-smi

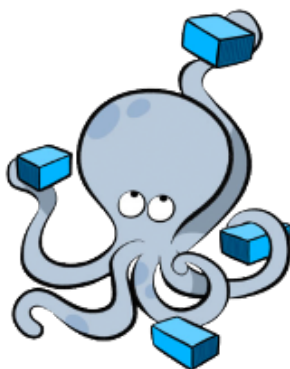


Figura 5: Logo de Docker Compose

4.3 Instalar Docker Compose

Docker Compose es una herramienta para definir y ejecutar aplicaciones Docker multi-contenedor utilizando archivos YAML. Facilita la gestión de entornos complejos con múltiples servicios interconectados.

Basado en: <https://docs.docker.com/compose/install/>

```
1 # Descargar Docker Compose
2 sudo curl -L "https://github.com/docker/compose/releases/download/v2
   .24.0/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/
   docker-compose
3
4 # Hacer ejecutable
5 sudo chmod +x /usr/local/bin/docker-compose
6
7 # Verificar
8 docker-compose --version
```

Listing 4: Instalación de Docker Compose

4.4 Instalar kubectl



Figura 6: Logo de kubectl

kubectl es la herramienta de línea de comandos oficial para interactuar con clústeres de Kubernetes, permitiendo desplegar aplicaciones, inspeccionar recursos y gestionar el clúster.

Basado en: <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/>

```

1 # Descargar la ultima version
2 curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/
   release/stable.txt)/bin/linux/amd64/kubectl"
3
4 # Hacer ejecutable
5 chmod +x kubectl
6
7 # Mover a PATH
8 sudo mv kubectl /usr/local/bin/
9
10 # Verificar
11 kubectl version --client

```

Listing 5: Instalación de kubectl

```
Client Version: v1.34.0
```

Figura 7: Salida del comando kubectl version --client

```
Client Version: v1.34.0
```

Figura 8: Salida del comando kubectl version --client

4.5 Instalar containerd



Figura 9: Logo de containerd

containerd es un runtime de contenedores de alto rendimiento, utilizado por Kubernetes como CRI (Container Runtime Interface) para gestionar el ciclo de vida de contenedores.

Basado en: <https://containerd.io/docs/getting-started/>

```

1 # Instalar dependencias
2 sudo apt-get update
3 sudo apt-get install -y ca-certificates curl gnupg lsb-release
4
5 # Agregar clave GPG
6 sudo mkdir -p /etc/apt/keyrings
7 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
   dearmor -o /etc/apt/keyrings/docker.gpg

```

```
8
9 # Agregar repositorio
10 echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/
    keyrings/docker.gpg] https://download.docker.com/linux/ubuntu$(
    lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.
    list > /dev/null
11
12 # Instalar containerd
13 sudo apt-get update
14 sudo apt-get install -y containerd.io
15
16 # Verificar
17 containerd --version
```

Listing 6: Instalación de containerd

```
containerd github.com/containerd/containerd v1.7.0
```

Figura 10: Salida del comando containerd --version

4.6 Instalar etcd



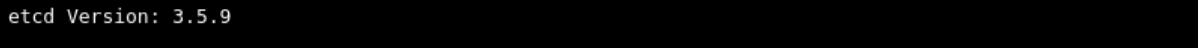
Figura 11: Logo de etcd

etcd es un almacén de clave-valor distribuido y consistente, utilizado por Kubernetes para almacenar datos de configuración y estado del clúster.

Basado en: <https://etcd.io/docs/v3.5/install/>

```
1 # Descargar etcd
2 ETCD_VER=v3.5.9
3 DOWNLOAD_URL=https://github.com/etcd-io/etcd/releases/download
4 curl -L ${DOWNLOAD_URL}/${ETCD_VER}/etcd-${ETCD_VER}-linux-amd64.tar.gz
    -o /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz
5
6 # Extraer
7 tar xzvf /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz -C /tmp/
8 sudo mv /tmp/etcd-${ETCD_VER}-linux-amd64/etcd* /usr/local/bin/
9
10 # Verificar
11 etcd --version
```

Listing 7: Instalación de etcd



```
etcd Version: 3.5.9
```

Figura 12: Salida del comando `etcd -version`

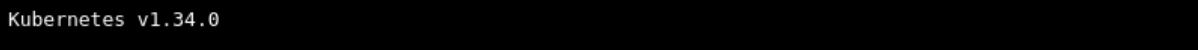
4.7 Instalar kubelet

kubelet es el agente principal que se ejecuta en cada nodo de Kubernetes, responsable de gestionar los contenedores y reportar el estado al plano de control.

Basado en: <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

```
1 # Descargar kubelet
2 KUBE_VER=$(curl -L -s https://dl.k8s.io/release/stable.txt)
3 curl -LO "https://dl.k8s.io/release/${KUBE_VER}/bin/linux/amd64/kubelet
4 "
5 # Hacer ejecutable
6 chmod +x kubelet
7
8 # Mover a PATH
9 sudo mv kubelet /usr/local/bin/
10
11 # Verificar
12 kubelet --version
```

Listing 8: Instalación de kubelet



```
Kubernetes v1.34.0
```

Figura 13: Salida del comando `kubelet -version`

Índice

5 Objetivos del Proyecto

Este proyecto tiene como objetivos principales proporcionar una configuración completa y profesional para crear un clúster de Kubernetes utilizando Kind con soporte para GPUs NVIDIA. Incluye scripts automatizados, documentación detallada y ejemplos prácticos para facilitar la implementación y el uso.

Los objetivos específicos se dividen en las siguientes categorías:

- **Configuración Completa:** Proporcionar una guía paso a paso para instalar y configurar un clúster Kind con soporte GPU.
- **Automatización:** Incluir scripts automatizados para simplificar la instalación y verificación.
- **Documentación Profesional:** Ofrecer documentación detallada con ejemplos y soluciones a problemas comunes.
- **Soporte para GPUs:** Habilitar el uso de GPUs NVIDIA en contenedores Kubernetes para aplicaciones de IA y computación intensiva.
- **Facilitación del Aprendizaje:** Democratizar el acceso a tecnologías avanzadas para desarrolladores, investigadores y estudiantes.

Para lograr estos objetivos, el proyecto se estructura en fases claras: desde la verificación de requisitos hasta la implementación, verificación y uso práctico del clúster.

6 Requisitos del Sistema

Antes de comenzar la implementación, es esencial verificar que su sistema cumpla con los requisitos mínimos. Esta sección detalla los prerequisites en hardware, software, red y permisos necesarios para un funcionamiento óptimo del clúster Kind con soporte GPU.

6.1 Hardware

Cuadro 1: Requisitos de Hardware

Requisito	Descripción
GPU NVIDIA	Una GPU NVIDIA compatible, como RTX 3060, necesaria para ejecutar cargas de trabajo con aceleración GPU en contenedores. Sin una GPU compatible, no se pueden aprovechar las capacidades de procesamiento paralelo para aplicaciones de IA o computación intensiva.
RAM	Al menos 8 GB de RAM para ejecutar el clúster Kind con varios nodos, incluyendo el plano de control y trabajadores. La RAM insuficiente puede causar fallos en la creación del clúster o lentitud en las operaciones.
CPU	Mínimo 4 núcleos de CPU para un rendimiento adecuado del clúster, permitiendo la ejecución simultánea de componentes de Kubernetes y aplicaciones. Menos núcleos pueden resultar en tiempos de respuesta lentos.

6.2 Software

Cuadro 2: Requisitos de Software

Requisito	Descripción
Docker	Versión 20.10+ requerida para ejecutar Kind y los contenedores del clúster. Docker actúa como el motor de contenedorización subyacente que Kind utiliza para crear nodos simulados.
Drivers NVIDIA	Versión 575.64.03+ con CUDA 12.9 para acceso a GPU. Estos drivers permiten la comunicación entre el sistema operativo y la GPU, esencial para aplicaciones que requieren CUDA.
Kernel Linux	Versión 5.4+ para soporte de paso a través de dispositivos GPU. Un kernel más antiguo puede no admitir el montaje directo de dispositivos GPU en contenedores.
curl/wget	Herramientas de línea de comandos para descargas de scripts, imágenes y paquetes desde internet, necesarias durante la instalación.

6.3 Red

Cuadro 3: Requisitos de Red

Requisito	Descripción
Puertos	Puertos 12741-12761 disponibles para exposición de servicios del clúster. Estos puertos se utilizan para acceder a aplicaciones desplegadas en el clúster desde el host local.
Internet	Acceso a internet para descarga de imágenes de contenedores y componentes de Kubernetes. Sin conexión, se requieren imágenes pre-descargadas o un registro local.

6.4 Permisos

Cuadro 4: Requisitos de Permisos

Requisito	Descripción
Sudo	Acceso sudo para instalación de Kind y herramientas del sistema. Permite ejecutar comandos con privilegios elevados necesarios para instalar paquetes y modificar configuraciones del sistema.
Grupo Docker	Membresía en el grupo Docker o permisos sudo para ejecutar comandos Docker. Sin esto, los comandos Docker fallarán debido a permisos insuficientes.

6.5 Verificación de Prerrequisitos

Ejecute `./scripts/validate.sh`¹ para verificar automáticamente que todos los requisitos del sistema estén cumplidos. Este script realiza comprobaciones de hardware, software y permisos, proporcionando un informe detallado de cualquier problema potencial.

¹En caso de no encontrar el script, puede crearlo usando el Anexo 2.

7 Instalación de Herramientas

Esta sección describe los pasos para instalar las herramientas necesarias.

****Nota:**** Para una instalación automatizada completa, incluyendo todas las dependencias y pruebas, utilice el script `mega_setup.sh` (requiere `sudo`). Consulte el Anexo 8 para más detalles.

7.1 Instalar Kind

Kind es la herramienta principal para crear y gestionar clústeres de Kubernetes locales usando Docker. Simplifica el desarrollo y pruebas al proporcionar un entorno reproducible.

Basado en: <https://kind.sigs.k8s.io/docs/user/quick-start/>



```
$ time kind create cluster
Creating cluster "kind" ...
✓ Ensuring node image (kindest/node:v1.16.3) 📁
✓ Preparing nodes 📦
✓ Writing configuration 📄
✓ Starting control-plane 🚦
✓ Installing CNI 🌐
✓ Installing StorageClass 💾
Set kubectl context to "kind-kind"
You can now use your cluster with:

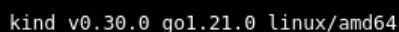
kubectl cluster-info --context kind-kind

Not sure what to do next? 😊 Check out https://kind.sigs.k8s.io/docs/user/quick-start/

real    0m21.890s
user    0m1.278s
sys     0m0.790s
```

Figura 14: Guía de inicio rápido de Kind

```
1 curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.30.0/kind-linux-amd64
2 chmod +x ./kind
3 sudo mv ./kind /usr/local/bin/kind
4 kind --version
```



```
kind v0.30.0 go1.21.0 linux/amd64
```

Figura 15: Salida del comando `kind --version`

8 Arquitectura del Clúster

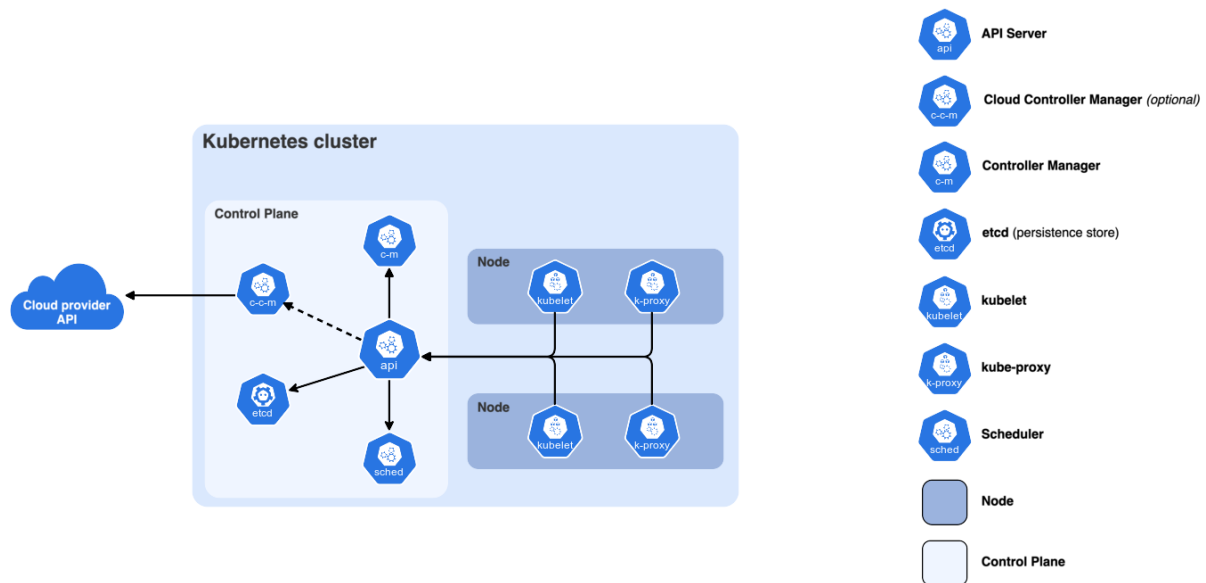


Figura 16: Arquitectura de Kubernetes

Esta configuración crea un cluster Kind con soporte para GPU mediante paso a través de dispositivos.

8.1 Componentes

8.1.1 Nodos

- **Plano de Control:** 1 node ejecutando componentes de control de Kubernetes.
- **Trabajadores:** 4 nodes para cargas de trabajo de aplicaciones.

8.1.2 Redes

- **CNI:** Kindnet (por defecto).
- **Puertos:** 12741-12761 expuestos en el plano de control para acceso externo.
- **Interna:** Red de pods 10.96.0.0/16.

8.1.3 Acceso a GPU

- **Método:** Montajes de dispositivos + montajes de bibliotecas.
- **Dispositivos:** /dev/nvidia* (GPU, UVM, modeset, ctl).
- **Bibliotecas:** libnvidia-ml.so.1, libcuda.so.1.
- **Etiquetas:** nvidia.com/gpu.present=true en todos los nodos.

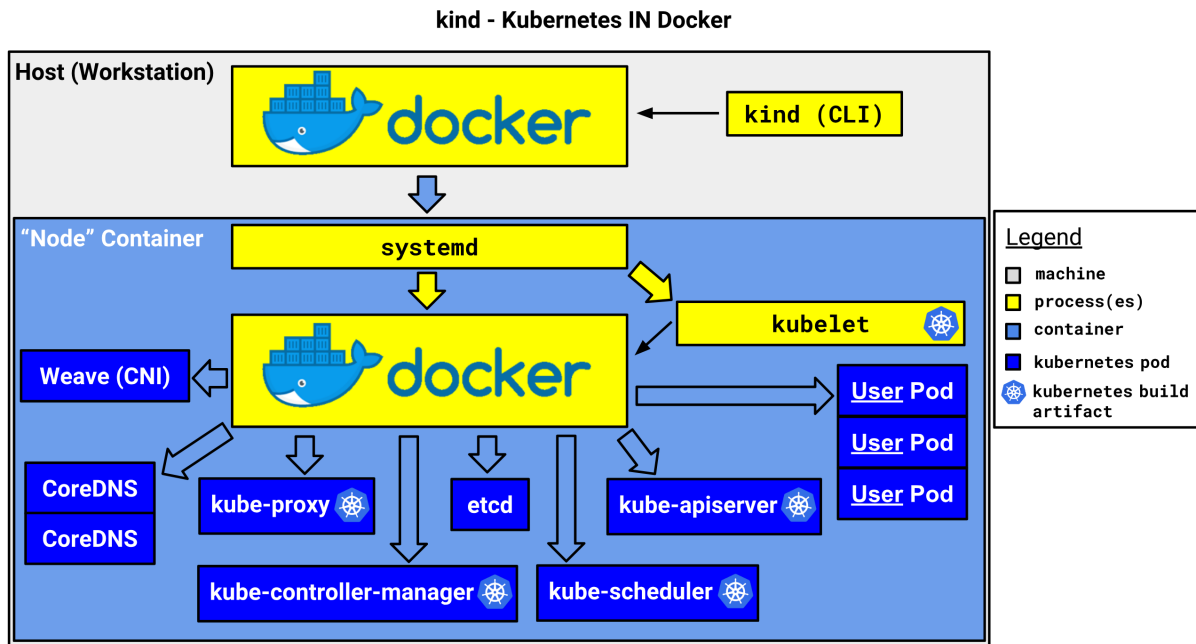


Figura 17: Diagrama del clúster Kind

8.1.4 Almacenamiento

- CSI: Clase de almacenamiento estándar (hostPath).

8.2 Seguridad

- Acceso root en contenedores (por defecto en Kind).
- Acceso a dispositivos GPU mediante montajes.
- Sin modificaciones RBAC.

8.3 Limitaciones

- Plano de control único para estabilidad.
- Programación de GPU mediante plugin de dispositivo (opcional).
- Sin volúmenes persistentes por defecto.

Con la arquitectura definida, el siguiente paso es la creación y configuración práctica del clúster Kind.

9 Creación y Configuración del Clúster

9.1 Creación del Clúster

Utilice el archivo `config/kind-config.yaml`² para crear el cluster:

```
1 kind create cluster --config config/kind-config.yaml\footnote{En caso  
de no encontrar el archivo, puede crearlo usando el Anexo 6.}
```

Qué crea:

- 1 nodo de plano de control.
- 4 nodos trabajadores.
- Puertos 12741-12761 expuestos en el plano de control.
- Dispositivos GPU montados en todos los nodos.
- Etiquetas de presencia de GPU.

9.2 Eliminación del Clúster

Si es necesario, elimine primero el cluster:

```
1 kind delete cluster
```

9.3 Verificación de Creación

Verifique que el cluster se haya creado correctamente:

```
1 kind get clusters  
2 kubectl cluster-info
```

9.4 Consideraciones Adicionales

- El clúster Kind es efímero por defecto; los datos se pierden al eliminar el clúster.
- Para persistencia, considere usar volúmenes `hostPath` o soluciones de almacenamiento externas.
- Monitoree el uso de recursos con herramientas como `k9s` o `kubectl top`.
- Para producción, evalúe clústeres reales en la nube o on-premises.

²En caso de no encontrar el archivo, puede crearlo usando el Anexo 6.

9.5 Configuración de GPU

9.6 Montajes de Dispositivos

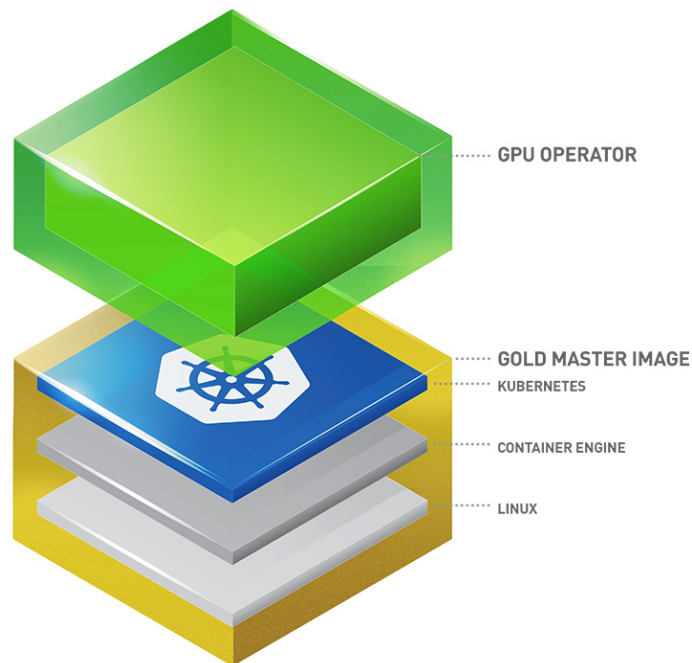


Figura 18: Operador GPU: Host a Kubernetes

La configuración monta:

- Dispositivos `/dev/nvidia*`.
- Bibliotecas NVIDIA (`libnvidia-ml.so.1`, `libcuda.so.1`).

9.7 Programación de GPU en Kubernetes

Instale el plugin de dispositivo NVIDIA:

```
1 kubectl apply -f https://raw.githubusercontent.com/NVIDIA/k8s-device-  
  plugin/v0.17.0/deployments/static/nvidia-device-plugin.yml
```

Verifique recursos GPU:

```
1 kubectl describe nodes | grep -A 5 nvidia.com/gpu
```

Una vez configurado el clúster y las GPUs, es crucial verificar que todo funcione correctamente antes de proceder al uso productivo.

10 Verificación y Pruebas

Verifique el estado del cluster:

```
1 kubectl get nodes
2 kubectl get nodes --show-labels
3 kubectl cluster-info
```

Listing 9: Verificación del estado del clúster

Todos los nodos deben estar **Ready** con etiquetas de GPU.

11 Pruebas

11.1 Validacion de Acceso a GPU

Ejecute comandos para probar el acceso a GPU:

```

1 # Copiar nvidia-smi a trabajadores
2 docker cp /usr/bin/nvidia-smi kind-worker:/usr/bin/nvidia-smi
3 docker cp /usr/bin/nvidia-smi kind-worker2:/usr/bin/nvidia-smi
4 docker cp /usr/bin/nvidia-smi kind-worker3:/usr/bin/nvidia-smi
5 docker cp /usr/bin/nvidia-smi kind-worker4:/usr/bin/nvidia-smi
6
7 # Probar
8 docker exec kind-worker nvidia-smi
9 docker exec kind-worker2 nvidia-smi
10 docker exec kind-worker3 nvidia-smi
11 docker exec kind-worker4 nvidia-smi

```

Listing 10: Validación de acceso a GPU en nodos trabajadores

Esperado: Información de GPU del host.

Thu Nov 13 08:58:20 2025

NVIDIA-SMI 575.64.03			Driver Version: 575.64.03			CUDA Version: 12.9		
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.	
						MIG	M.	
0	NVIDIA GeForce RTX 3060	Off	00000000:01:00.0	On				N/A
0%	39C	P8	14W / 170W	1861MiB / 12288MiB	30%	Default		N/A

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory	
	ID	ID				Usage	
0	N/A	N/A	5157	G	/usr/lib/xorg/Xorg	357MiB	
0	N/A	N/A	5390	G	/usr/bin/gnome-shell	36MiB	
0	N/A	N/A	14993	G	gnome-text-editor	99MiB	
0	N/A	N/A	1095659	G	...b/thunderbird/thunderbird-bin	9MiB	
0	N/A	N/A	1100426	G	...op/40/unpacked/gemini-desktop	35MiB	
0	N/A	N/A	1143357	G	.../7177/usr/lib/firefox/firefox	23MiB	
0	N/A	N/A	1215057	G	/opt/Lens/lens-desktop	14MiB	
0	N/A	N/A	1599955	G	/usr/bin/nautilus	668MiB	
0	N/A	N/A	1601175	G	...exec/xdg-desktop-portal-gnome	190MiB	
0	N/A	N/A	1744771	G	/usr/share/code/code	223MiB	
0	N/A	N/A	2700620	G	nextcloud	2MiB	
0	N/A	N/A	4095895	G	...ersion=20251030-124156.606000	48MiB	

Figura 19: Salida de nvidia-smi en worker

11.2 Prueba Automatizada

Utilice `setup.sh`³ que incluye pruebas de GPU.

Si las pruebas pasan exitosamente, el clúster está listo para uso. En caso de problemas, consulte la siguiente sección de solución de problemas.

³En caso de no encontrar el script, puede crearlo usando el Anexo 1.

12 Solución de Problemas

12.1 Creación del Clúster Falla

- Asegúrese de que Docker esté ejecutándose: `sudo systemctl start docker`.
- Verifique información de Docker: `docker info`.
- Si hay tiempo de espera, simplifique la configuración (reduzca nodos).

12.2 Nodos No Listos

- Espere 30-60 segundos después de la creación.
- Verifique CNI: `kubectl get pods -n kube-system`.

12.3 GPU No Detectada

- Verifique que el host tenga drivers NVIDIA: `nvidia-smi`.
- Verifique montajes en la configuración.
- Para programar GPU en Kubernetes, el plugin de dispositivo debe ejecutarse.

12.4 Problemas de Exposición de Puertos

- Los puertos 12741-12761 están expuestos en el plano de control.
- Use `docker ps` para verificar mapeos.

12.5 Errores del Plugin de Dispositivo

- "demasiados archivos abiertos": Aumente `ulimits` o omita el plugin.
- Use montajes directos para acceso a GPU.

12.6 Recrear Clúster

```
1 kind delete cluster
2 kind create cluster --config kind-config.yaml\footnote{En caso de no
  encontrar el archivo, puede crearlo usando el Anexo 6.}
```


13 Limpieza y Mantenimiento

```
1 ./scripts/cleanup.sh
2 # Or manually:
3 kind delete cluster
```

⁴

Para consultas comunes y consejos adicionales, consulte las preguntas frecuentes a continuación.

⁴En caso de no encontrar el script, puede crearlo usando el Anexo 3.

14 Preguntas Frecuentes

14.1 General

¿Por qué Kind?

Kind proporciona clusteres ligeros de Kubernetes para desarrollo/pruebas con soporte para GPU.

¿Por qué no minikube?

Kind soporta mejor clusteres multi-nodo y paso a través de GPU.

14.2 Configuración

¿La creación del cluster se agota?

Reduzca nodos en la configuración o aumente el tiempo de espera. Use 1 plano de control + menos trabajadores.

¿Docker no encontrado?

Asegúrese de que el daemon Docker esté ejecutándose: `sudo systemctl start docker`.

¿GPU no detectada?

Verifique drivers del host con `nvidia-smi`. Asegúrese de que los montajes en la configuración coincidan con las rutas del host.

14.3 GPU

¿El plugin de dispositivo falla?

Use montajes directos para acceso. El plugin requiere drivers NVIDIA en los nodos (no presentes).

¿Cómo programar pods GPU?

Use nodeSelector: `nvidia.com/gpu.present: "true"`.

¿Múltiples GPUs?

La configuración monta `/dev/nvidia0`; ajuste para más GPUs.

14.4 Uso

¿Cómo acceder a servicios?

Use puertos expuestos 12741-12761 en localhost.

¿Almacenamiento persistente?

Use volúmenes `hostPath` o agregue drivers CSI.

¿Agregar más nodos?

No soportado; recree el cluster con nueva configuración.

14.5 Solucion de Problemas

¿Nodos no Ready?

Espere 1-2 minutos. Verifique pods CNI: `kubectl get pods -n kube-system`.

¿kubectl conexión rechazada?

Ejecute `kind export kubeconfig` o verifique estado del cluster con `kind get clusters`.

Para ilustrar el uso práctico del clúster, a continuación se presentan ejemplos detallados de despliegues y operaciones comunes.

15 Ejemplos de Uso

15.1 Ejecutar Pod GPU

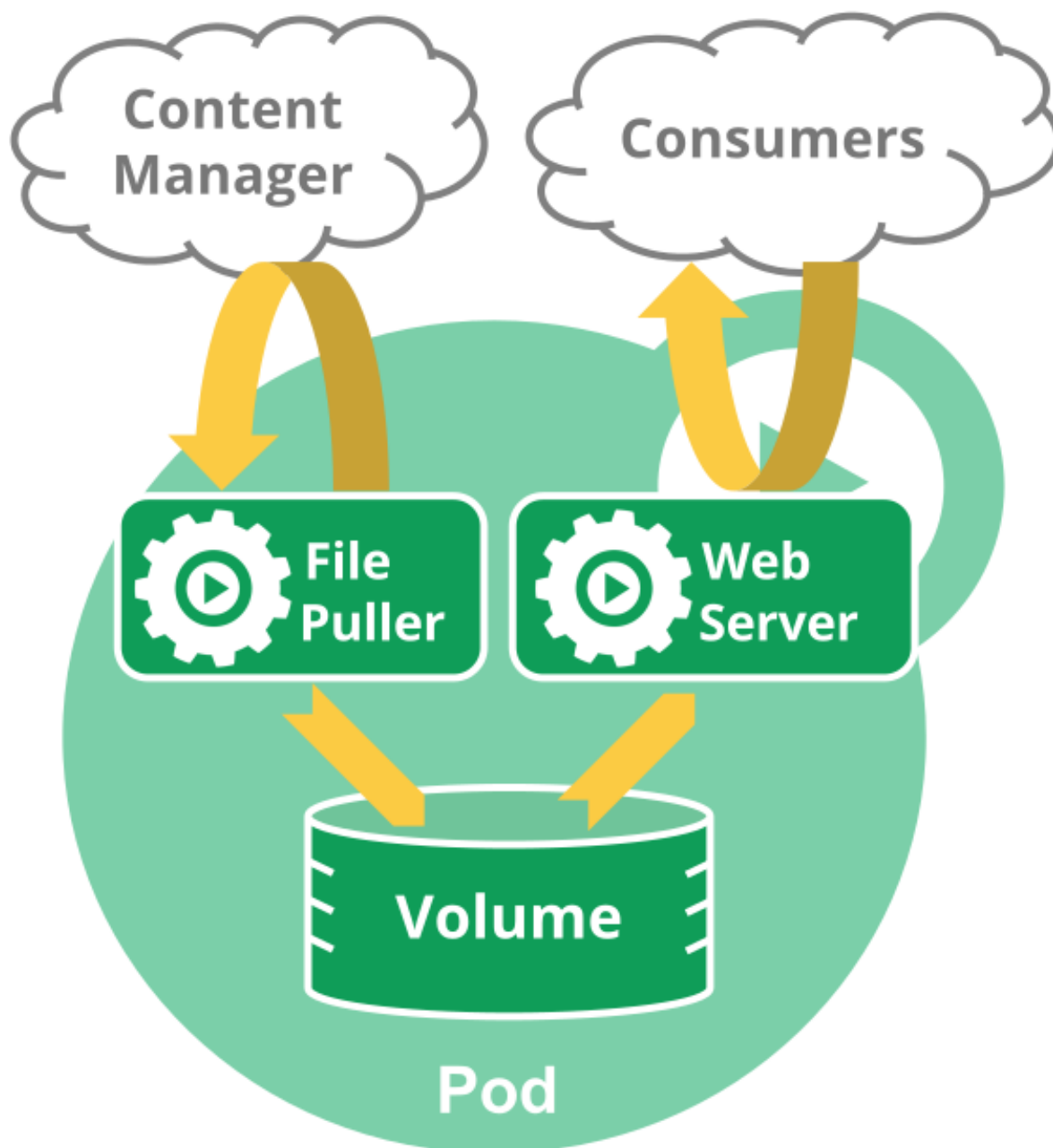


Figura 20: Diagrama de un Pod en Kubernetes

```
1 kubectl apply -f examples/gpu-pod-example.yaml
2 kubectl logs gpu-pod
```

El archivo YAML del ejemplo:

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: gpu-pod
```

```
5 spec:
6   nodeSelector:
7     nvidia.com/gpu.present: "true"
8   containers:
9   - name: gpu-container
10     image: nvidia/cuda:12.2.0-base-ubuntu22.04
11     command: ["nvidia-smi"]
12     resources:
13       limits:
14         nvidia.com/gpu: 1 # Solicitar 1 GPU (si el plugin funciona)
15     restartPolicy: Never
```

15.2 Ejemplo de Despliegue de una Aplicación Simple

Este ejemplo demuestra cómo desplegar una aplicación web simple en el clúster Kind. Sirve para verificar que el clúster está funcionando correctamente y para practicar comandos básicos de Kubernetes.

```
1 kubectl create deployment hello-world --image=gcr.io/google-samples/
  hello-app:1.0
2 kubectl expose deployment hello-world --type=LoadBalancer --port=8080
3 kubectl get services
```

15.3 Ejemplo de Uso de GPU para Computación Científica

Este ejemplo muestra cómo ejecutar un contenedor que utiliza CUDA para cálculos científicos. Sirve para validar el soporte de GPU en el clúster y demostrar el uso de recursos GPU en aplicaciones de alto rendimiento.

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: cuda-vector-add
5 spec:
6   nodeSelector:
7     nvidia.com/gpu.present: "true"
8   containers:
9   - name: cuda-vector-add
10     image: nvcr.io/nvidia/k8s/cuda-sample:vectoradd-cuda10.2
11     resources:
12       limits:
13         nvidia.com/gpu: 1
14     restartPolicy: Never
```

15.4 Ejemplo de Escalado de Pods

Este ejemplo ilustra cómo escalar una aplicación para manejar más carga. Sirve para aprender sobre la escalabilidad horizontal en Kubernetes y cómo ajustar recursos dinámicamente.

```
1 kubectl scale deployment hello-world --replicas=3
2 kubectl get pods
```

15.5 Ejemplo de Monitoreo con k9s

Este ejemplo explica cómo usar k9s para monitorear el estado del clúster en tiempo real. Sirve para facilitar la observación y debugging de aplicaciones en Kubernetes sin comandos complejos.

```
1 k9s
2 # Dentro de k9s: presiona 'p' para ver pods, 'd' para deployments
```

15.6 Ejemplo de Despliegue con ArgoCD

Este ejemplo muestra cómo desplegar una aplicación usando GitOps con ArgoCD. Sirve para automatizar despliegues continuos y mantener la sincronización entre código y clúster.

```
1 kubectl apply -f argocd-app.yaml
```

Donde argocd-app.yaml contiene:

```
1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: my-app
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/my-org/my-app
10    targetRevision: HEAD
11    path: .
12   destination:
13     server: https://kubernetes.default.svc
14     namespace: default
```

15.7 Ejemplo de Configuración de Recursos GPU Avanzada

Este ejemplo demuestra cómo configurar límites y solicitudes de GPU para múltiples contenedores. Sirve para optimizar el uso de recursos GPU en aplicaciones complejas con múltiples componentes.

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: multi-gpu-pod
5 spec:
6   nodeSelector:
```

```
7   nvidia.com/gpu.present: "true"
8   containers:
9   - name: gpu-container-1
10     image: nvidia/cuda:12.2.0-base-ubuntu22.04
11     resources:
12       limits:
13         nvidia.com/gpu: 1
14       requests:
15         nvidia.com/gpu: 1
16   - name: gpu-container-2
17     image: nvidia/cuda:12.2.0-base-ubuntu22.04
18     resources:
19       limits:
20         nvidia.com/gpu: 1
21       requests:
22         nvidia.com/gpu: 1
23   restartPolicy: Never
```

15.8 Ejemplo de Debugging de Pods con GPU

Este ejemplo muestra comandos para diagnosticar problemas en pods que usan GPU. Sirve para solucionar issues comunes como fallos de asignación de GPU o errores de contenedores.

```
1 kubectl describe pod gpu-pod
2 kubectl logs gpu-pod
3 kubectl exec -it gpu-pod -- nvidia-smi
```

15.9 Ejemplo de Backup y Restauración de Configuración

Este ejemplo ilustra cómo hacer backup de la configuración del clúster. Sirve para preservar configuraciones importantes y facilitar la recuperación en caso de fallos.

```
1 kubectl get all --all-namespaces -o yaml > cluster-backup.yaml
2 # Para restaurar: kubectl apply -f cluster-backup.yaml
```

Además de los ejemplos básicos, el proyecto incluye herramientas adicionales para mejorar la experiencia de gestión del clúster.

16 Herramientas Adicionales

16.0.1 Instalacion de ArgoCD



Figura 21: Logo de ArgoCD

ArgoCD es una herramienta GitOps para Kubernetes.

```
1 kubectl create namespace argocd
2 kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/
  argo-cd/stable/manifests/install.yaml
```

Listing 11: Instalación de ArgoCD en el clúster

Espere a que los pods estén listos:

```
1 kubectl get pods -n argocd
```

Listing 12: Verificación de pods de ArgoCD

Acceda a ArgoCD:

```
1 kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Listing 13: Acceso a la interfaz web de ArgoCD

Abra <https://localhost:8080>

Credenciales:

- Usuario: admin
- Contraseña: `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath=".data.password" | base64 -d`

16.0.2 Instalacion de k9s

k9s es una interfaz de usuario basada en terminal para gestionar clústeres de Kubernetes de manera eficiente. Proporciona una experiencia interactiva para navegar por recursos, logs, pods y más, sin necesidad de comandos complejos de kubectl.

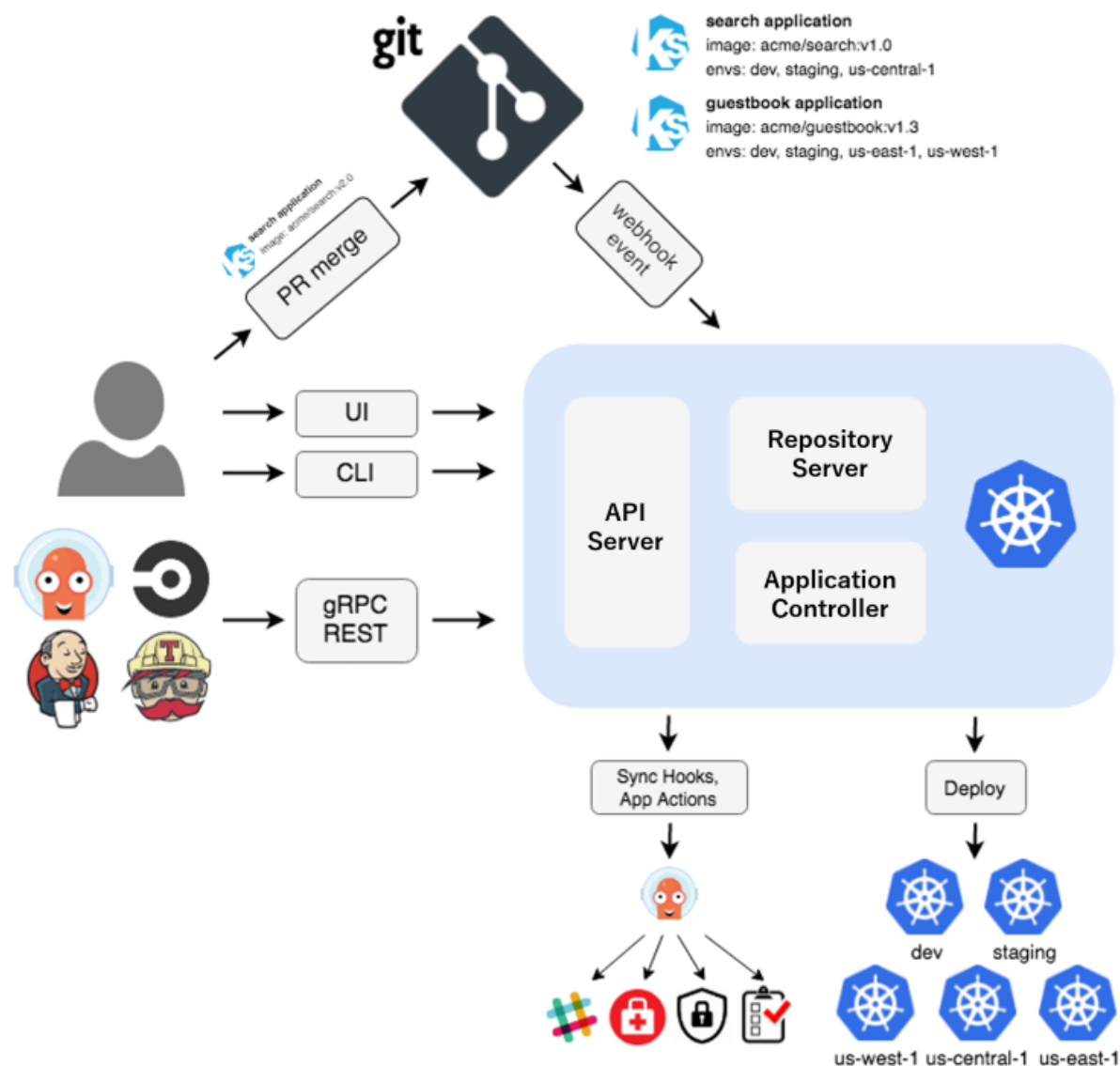


Figura 22: Arquitectura de ArgoCD

- **Ventajas:** Navegación rápida con atajos de teclado, vistas en tiempo real, y comandos integrados para operaciones comunes.
- **Uso:** Ideal para monitoreo y debugging en entornos de desarrollo.

Instalación:

```

1  # Descargar la última versión
2  curl -Lo ./k9s.tar.gz https://github.com/derailed/k9s/releases/latest/download/k9s_Linux_amd64.tar.gz
3
4  # Extraer
5  tar -xzf k9s.tar.gz
6
7  # Hacer ejecutable
8  chmod +x k9s
9
10 # Mover a directorio en PATH

```

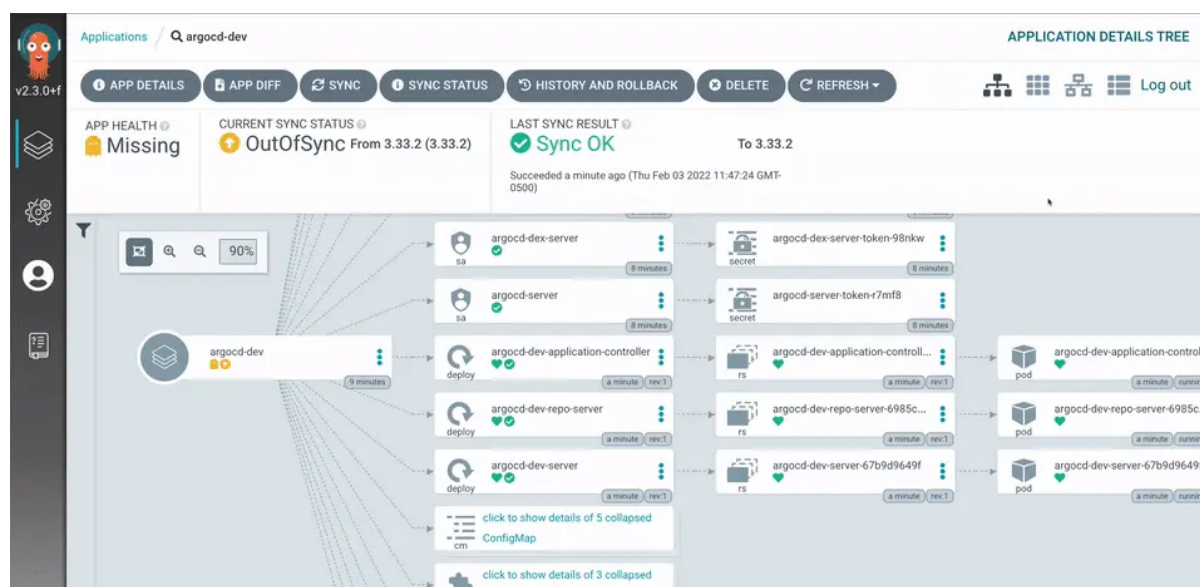


Figura 23: Interfaz de usuario de ArgoCD

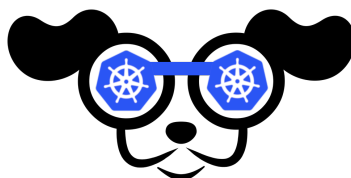


Figura 24: Logo de k9s

```
11 mkdir -p ~/bin
12 mv k9s ~/bin/
```

Listing 14: Descarga e instalación de k9s

Agregar a PATH (temporalmente):

```
1 source scripts/set_path.sh
```

Listing 15: Configuración temporal de PATH para k9s

O permanentemente:

```
1 echo 'export PATH="$PATH:$HOME/bin"' >> ~/.bashrc
2 source ~/.bashrc
```

Listing 16: Configuración permanente de PATH para k9s

5

Ejecutar k9s:

```
1 k9s
```

Listing 17: Ejecución de k9s

⁵En caso de no encontrar el script, puede crearlo usando el Anexo 5.

The screenshot shows the k9s terminal interface. At the top left, it displays the current context: **Context: minikube**, **Cluster: minikube**, **User: minikube**, **K9s Rev: dev**, **K8s Rev: v1.17.3**, **CPU: 5%**, and **MEM: 17%**. To the right of this, there are navigation and action shortcuts: **<0> all**, **<1> kube-system**, **<2> default**, **<a> Attach**, **<ctrl-d> Delete**, **<d> Describe**, **<e> Edit**, **<ctrl-k> Kill**, **<l> Logs**, **<ctrl-j> Logs (jq)**, **<ctrl-l> Logs <Stern>**, **<shift-l> Logs Previous**, **<shift-f> Port-Forward**, **<s> Shell**, and **<y> YAML**. On the far right, there is a dashed box containing a diagram of a network topology. Below this, a table titled **Pods(all)[23]** lists the pods in the cluster. The table has columns for **NAMESPACE**, **NAME**, **READY**, **RESTART**, **STATUS**, **CPU**, **MEM**, **%CPU/R**, **%MEM/R**, **%CPU/L**, **%MEM/L**, **IP**, and **NODE**. The pods listed include **hello-1582785780-lsrttd**, **hello-1582785849-rq8h5**, **hello-1582785908-4z8kf**, **jaeger-5bbc8c887-cmj77**, **nginx**, **nginx-6fbbddc48c-5kv5p**, **nginx-6fbbddc48c-7xn7j**, **nginx-6fbbddc48c-bmqqj**, **nginx-6fbbddc48c-jf944**, **nginx-6fbbddc48c-xwjnb**, **coredns-6955765f44-2pkvx**, **coredns-6955765f44-wr88k**, **etcd-minikube**, **fluentd-elasticsearch-vnt25**, **kube-apiserver-minikube**, **kube-controller-manager-minikube**, **kube-proxy-sqs9s**, **kube-scheduler-minikube**, **metrics-server-6754dbc9df-t8x2n**, **metrics-server-6754dbc9df-tz7kh**, **storage-provisioner**, **kubernetes-dashboard**, and **kubernetes-dashboard-79d9cd965-wb2vz**. At the bottom left, there are buttons for **<pulses>** and **<pod>**.

Figura 25: Captura de pantalla de k9s

k9s detecta automáticamente el contexto kubeconfig actual y se conecta al clúster. Use las teclas de navegación para explorar recursos como pods, servicios y deployments. Presione ? para ver la ayuda integrada.

16.0.3 Integración con Lens

Lens es una interfaz gráfica (GUI) para gestionar clústeres de Kubernetes, proporcionando una experiencia visual intuitiva similar a un IDE. Permite visualizar recursos, editar YAML, ver logs en tiempo real y gestionar aplicaciones sin necesidad de línea de comandos.

- **Ventajas:** Interfaz amigable para principiantes, integración con Helm, y soporte para múltiples clústeres.
- **Uso:** Perfecto para usuarios que prefieren una interfaz gráfica sobre la terminal.

Instalación de Lens:

Basado en: <https://docs.k8slens.dev/getting-started/install-lens/>

```

1 # Descargar el paquete .deb (para Ubuntu/Debian)
2 curl -LO https://api.k8slens.dev/binaries/Lens-2023.12.121249-latest.
   amd64.deb
3
4 # Instalar
5 sudo dpkg -i Lens-2023.12.121249-latest.amd64.deb
6
7 # Resolver dependencias si es necesario
8 sudo apt install -f

```

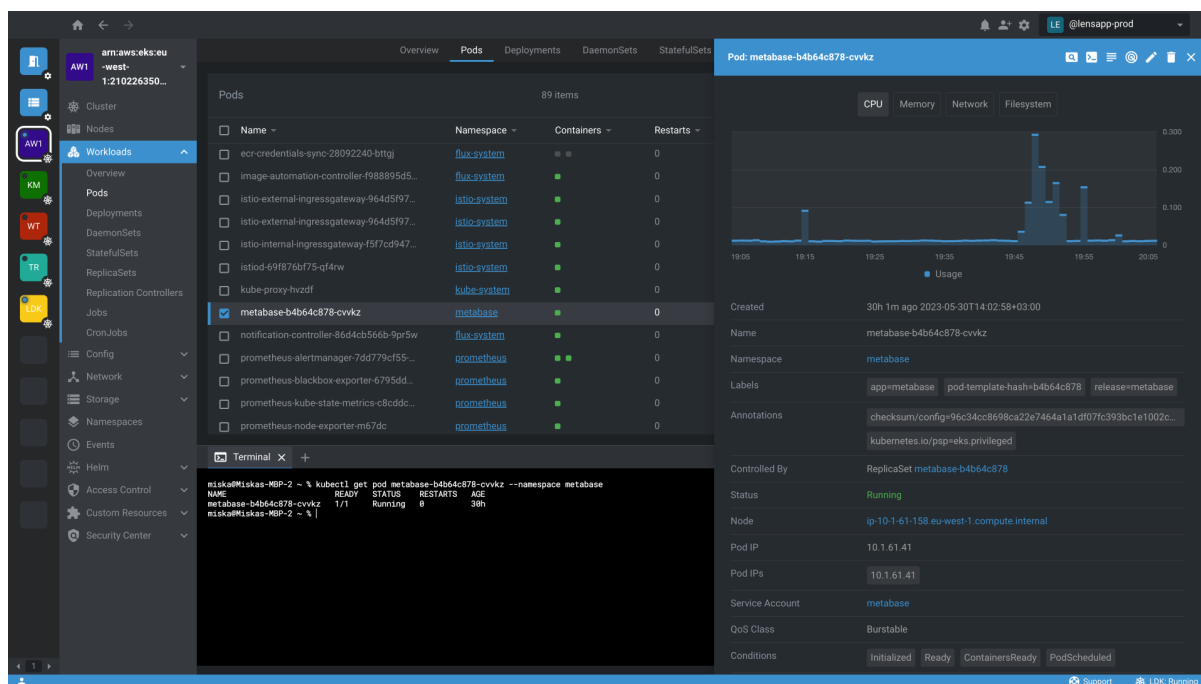


Figura 26: Captura de pantalla de Lens

Alternativamente, descargar desde el sitio web oficial y seguir las instrucciones para Linux.

Configuración con el clúster Kind:

Primero, obtener el archivo de configuración del clúster:

```
1 kind get kubeconfig > config/kubeconfig.yaml
```

En Lens:

1. Abra Lens.
2. Haga clic en "Add Cluster" o "Import Cluster".
3. Seleccione "From kubeconfig" y elija `config/kubeconfig.yaml`.
4. El clúster aparecerá en la lista; selecciónelo para conectarse.

Una vez conectado, puede explorar nodos, pods, servicios y más a través de la interfaz gráfica. Lens también permite instalar extensiones y gestionar Helm charts.

17 Registro de Cambios

Todas las modificaciones notables a este proyecto se documentarán en este archivo.

El formato se basa en [Keep a Changelog](#), y este proyecto adhiere a [Versionado Semántico](#).

17.1 [1.1.0] - 2025-11-13

17.1.1 Agregado

Agregado Mega setup script (mega_setup.sh) para configuración automatizada completa.

17.1.2 Características

- Validación de acceso GPU en todos los trabajadores.
- Documentación profesional.
- Scripts de limpieza.

17.2 [1.0.0] - 2025-11-12

17.2.1 Agregado

Agregado Configuración inicial para clúster Kind con soporte GPU.
Scripts automatizados para instalación y pruebas.
Configuración para 1 plano de control + 4 trabajadores.
Exposición de puertos (12741-12761).
Montaje de dispositivos GPU y etiquetado.
Guías de validación y solución de problemas.
Makefile para automatización.
Ejemplo de pod GPU en YAML.

17.2.2 Características

- Validación de acceso GPU en todos los trabajadores.
- Documentación profesional.
- Scripts de limpieza.

18 Licencia

Este proyecto está licenciado bajo la Licencia MIT - consulte el archivo LICENSE para detalles.

MIT License

Copyright (c) 2025 Kind GPU Setup

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

19 Apéndices

19.1 Versión de Componentes

Componente	Versión	Descripción
Kind	v0.30.0	Herramienta para ejecutar clusters Kubernetes locales usando contenedores Docker.
Kubernetes	v1.34.0 (kindest/node)	Plataforma de orquestación de contenedores utilizada en el cluster.
Driver NVIDIA	575.64.03	Controlador para GPUs NVIDIA, necesario para el acceso a hardware GPU.
CUDA	12.9	Plataforma de computación paralela y modelo de programación de NVIDIA.
Plugin de Dispositivo NVIDIA	v0.17.0	Plugin de Kubernetes para gestionar recursos GPU en pods.

19.2 Estructura del Proyecto

```

1 kind_use/
2 - argocd-app.yaml          # Configuración de aplicación ArgoCD
3 - config/
4   - kind-config.yaml      # Configuración del cluster
5   - kubeconfig.yaml      # Kubeconfig para herramientas externas
6 - docs/
7   - documentation.tex     # Documentación en LaTeX
8   - documentation.pdf     # Documentación compilada
9   - README.md             # Esta guía
10  - architecture.md       # Arquitectura del sistema
11  - requirements.md        # Prerrequisitos
12  - FAQ.md                # Preguntas frecuentes
13  - troubleshooting.md    # Guía de solución de problemas
14  - CHANGELOG.md          # Historial de cambios
15  - *.txt                 # Archivos de referencia de comandos
16  - resources/            # Recursos (imágenes, resultados)
17 - examples/
18   - gpu-pod-example.yaml  # Ejemplo de pod GPU
19 - scripts/
20   - setup.sh              # Configuración automatizada básica
21   - cleanup.sh            # Script de limpieza
22   - validate.sh           # Verificación de prerrequisitos
23   - monitoring.sh         # Monitoreo del cluster
24   - set_path.sh           # Configuración de PATH
25   - mega_setup.sh         # Configuración automatizada completa
26   - cluster_manager.py    # Gestor de cluster en Python
27   - gpu_monitor.py        # Monitor de recursos GPU
28 - .gitignore              # Reglas de ignorar Git

```

```
29 - LICENSE                # Licencia MIT  
30 - Makefile              # Objetivos de automatizacion  
31 - versions.txt          # Informacion de versiones
```

Listing 18: Estructura del proyecto

20 Anexos

Este apartado contiene los scripts y archivos de configuración utilizados en la documentación. Cada anexo incluye el contenido completo del archivo correspondiente, con explicaciones de su propósito y uso.

20.1 16.1: Script setup.sh

Este script automatiza la instalación y configuración de Docker, NVIDIA Container Toolkit, kubectl, containerd, etcd, kubelet y Kind, facilitando la preparación del entorno para el cluster Kubernetes local.

```

1  #!/bin/bash
2
3  # Setup script for Kind cluster with GPU
4
5  echo "Installing Kind..."
6  curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.30.0/kind-linux-amd64
7  chmod +x ./kind
8  sudo mv ./kind /usr/local/bin/kind
9
10 echo "Creating cluster..."
11 kind create cluster --config kind-config.yaml\footnote{En caso de no
    encontrar el archivo, puede crearlo usando el Anexo 6.}
12
13 echo "Verifying cluster..."
14 kubectl get nodes
15
16 echo "Setting up GPU labels..."
17 kubectl label node kind-control-plane nvidia.com/gpu.present=true
18 kubectl label node kind-worker kind-worker2 kind-worker3 kind-worker4
    nvidia.com/gpu.present=true
19
20 echo "Installing NVIDIA device plugin..."
21 kubectl apply -f https://raw.githubusercontent.com/NVIDIA/k8s-device-
    plugin/v0.17.0/deployments/static/nvidia-device-plugin.yml
22
23 echo "Testing GPU on workers..."
24 docker cp /usr/bin/nvidia-smi kind-worker:/usr/bin/nvidia-smi
25 docker cp /usr/bin/nvidia-smi kind-worker2:/usr/bin/nvidia-smi
26 docker cp /usr/bin/nvidia-smi kind-worker3:/usr/bin/nvidia-smi
27 docker cp /usr/bin/nvidia-smi kind-worker4:/usr/bin/nvidia-smi
28
29 docker exec kind-worker nvidia-smi
30 docker exec kind-worker2 nvidia-smi
31 docker exec kind-worker3 nvidia-smi
32 docker exec kind-worker4 nvidia-smi
33
34 echo "Setup complete!"

```

Este script verifica que todos los prerrequisitos del sistema estén cumplidos antes de proceder con la creación del cluster, incluyendo la presencia de herramientas instaladas y configuraciones correctas.

20.2 16.2: Script validate.sh

```

1  #!/bin/bash
2
3  # Validation script for prerequisites
4
5  echo "Validating prerequisites..."
6
7  # Check Docker
8  if ! docker info > /dev/null 2>&1; then
9      echo "ERROR: Docker not running or not installed"
10     exit 1
11 fi
12 echo "OK: Docker OK"
13
14 # Check NVIDIA
15 if ! nvidia-smi > /dev/null 2>&1; then
16     echo "ERROR: NVIDIA GPU/drivers not available"
17     exit 1
18 fi
19 echo "OK: NVIDIA GPU OK"
20
21 # Check sudo
22 if ! sudo -n true 2>/dev/null; then
23     echo "WARNING: Sudo may require password"
24 fi
25
26 # Check if Kind exists
27 if command -v kind > /dev/null 2>&1; then
28     echo "OK: Kind installed: $(kind --version)"
29 else
30     echo "WARNING: Kind not installed, run install_kind.txt"
31 fi
32
33 # Check kubectl
34 if command -v kubectl > /dev/null 2>&1; then
35     echo "OK: kubectl installed"
36 else
37     echo "WARNING: kubectl not installed"
38 fi
39
40 echo "Validation complete!"

```

20.3 16.3: Script cleanup.sh

```

1  #!/bin/bash
2
3  # Cleanup script
4
5  echo "Deleting cluster..."
6  kind delete cluster
7
8  echo "Removing Kind binary..."
9  sudo rm /usr/local/bin/kind
10

```

```
11 echo "Cleanup complete!"
```

20.4 16.4: Script monitoring.sh

```
1 #!/bin/bash
2
3 # Monitoring script for cluster status
4
5 echo "=== Cluster Status ==="
6 kubectl get nodes -o wide
7
8 echo -e "\n=== Pod Status ==="
9 kubectl get pods -A --no-headers | wc -l
10 kubectl get pods -A | grep -v Running
11
12 echo -e "\n=== GPU Resources ==="
13 kubectl describe nodes | grep -A 5 "nvidia.com/gpu"
14
15 echo -e "\n=== Exposed Ports ==="
16 docker ps | grep kind-control-plane | grep -o "127[0-9]*->"
17
18 echo -e "\n=== Disk Usage ==="
19 df -h | grep -E "(docker|overlay)"
20
21 echo -e "\n=== Disk Usage ==="
22 df -h | grep -E "(docker|overlay)"
23
24 echo "Monitoring complete."
```

20.5 16.5: Script set_path.sh

```
1 #!/bin/bash
2
3 # Script to add ~/bin to PATH
4
5 export PATH="$PATH:$HOME/bin"
6 echo "Added ~/bin to PATH. Run 'source set_path.sh' or add to ~/.bashrc"
```

20.6 16.6: Archivo kind-config.yaml

```
1 kind: Cluster
2 apiVersion: kind.x-k8s.io/v1alpha4
3 nodes:
4 - role: control-plane
5   extraPortMappings:
6     - containerPort: 80
7       hostPort: 12741
8       listenAddress: "127.0.0.1"
9     - containerPort: 81
10      hostPort: 12742
11      listenAddress: "127.0.0.1"
12     - containerPort: 82
13      hostPort: 12743
14      listenAddress: "127.0.0.1"
15     - containerPort: 83
16      hostPort: 12744
17      listenAddress: "127.0.0.1"
18     - containerPort: 84
19      hostPort: 12745
20      listenAddress: "127.0.0.1"
21     - containerPort: 85
22      hostPort: 12746
23      listenAddress: "127.0.0.1"
24     - containerPort: 86
25      hostPort: 12747
26      listenAddress: "127.0.0.1"
27     - containerPort: 87
28      hostPort: 12748
29      listenAddress: "127.0.0.1"
30     - containerPort: 88
31      hostPort: 12749
32      listenAddress: "127.0.0.1"
33     - containerPort: 89
34      hostPort: 12750
35      listenAddress: "127.0.0.1"
36     - containerPort: 90
37      hostPort: 12751
38      listenAddress: "127.0.0.1"
39     - containerPort: 91
40      hostPort: 12752
41      listenAddress: "127.0.0.1"
42     - containerPort: 92
43      hostPort: 12753
44      listenAddress: "127.0.0.1"
45     - containerPort: 93
46      hostPort: 12754
47      listenAddress: "127.0.0.1"
48     - containerPort: 94
49      hostPort: 12755
50      listenAddress: "127.0.0.1"
51     - containerPort: 95
52      hostPort: 12756
53      listenAddress: "127.0.0.1"
54     - containerPort: 96
55      hostPort: 12757
56      listenAddress: "127.0.0.1"
```

```

57 - containerPort: 97
58   hostPort: 12758
59   listenAddress: "127.0.0.1"
60 - containerPort: 98
61   hostPort: 12759
62   listenAddress: "127.0.0.1"
63 - containerPort: 99
64   hostPort: 12760
65   listenAddress: "127.0.0.1"
66 - containerPort: 100
67   hostPort: 12761
68   listenAddress: "127.0.0.1"
69 extraMounts:
70 - hostPath: /dev/nvidia0
71   containerPath: /dev/nvidia0
72 - hostPath: /dev/nvidiactl
73   containerPath: /dev/nvidiactl
74 - hostPath: /dev/nvidiamodeset
75   containerPath: /dev/nvidiamodeset
76 - hostPath: /dev/nvidia-uvm
77   containerPath: /dev/nvidia-uvm
78 - hostPath: /dev/nvidia-uvm-tools
79   containerPath: /dev/nvidia-uvm-tools
80 - hostPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
81   containerPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
82 - hostPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
83   containerPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
84 kubeadmConfigPatches:
85 - |
86   kind: InitConfiguration
87   nodeRegistration:
88     kubeletExtraArgs:
89       node-labels: "nvidia.com/gpu.present=true"
90 - role: worker
91   extraMounts:
92   - hostPath: /dev/nvidia0
93     containerPath: /dev/nvidia0
94   - hostPath: /dev/nvidiactl
95     containerPath: /dev/nvidiactl
96   - hostPath: /dev/nvidiamodeset
97     containerPath: /dev/nvidiamodeset
98   - hostPath: /dev/nvidia-uvm
99     containerPath: /dev/nvidia-uvm
100 - hostPath: /dev/nvidia-uvm-tools
101   containerPath: /dev/nvidia-uvm-tools
102 - hostPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
103   containerPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
104 - hostPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
105   containerPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
106 - role: worker
107   extraMounts:
108   - hostPath: /dev/nvidia0
109     containerPath: /dev/nvidia0
110   - hostPath: /dev/nvidiactl
111     containerPath: /dev/nvidiactl
112   - hostPath: /dev/nvidiamodeset
113     containerPath: /dev/nvidiamodeset
114   - hostPath: /dev/nvidia-uvm

```

```

115     containerPath: /dev/nvidia-uvm
116 - hostPath: /dev/nvidia-uvm-tools
117     containerPath: /dev/nvidia-uvm-tools
118 - hostPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
119     containerPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
120 - hostPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
121     containerPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
122 - role: worker
123   extraMounts:
124   - hostPath: /dev/nvidia0
125     containerPath: /dev/nvidia0
126   - hostPath: /dev/nvidiactl
127     containerPath: /dev/nvidiactl
128   - hostPath: /dev/nvidiamodeset
129     containerPath: /dev/nvidiamodeset
130   - hostPath: /dev/nvidia-uvm
131     containerPath: /dev/nvidia-uvm
132   - hostPath: /dev/nvidia-uvm-tools
133     containerPath: /dev/nvidia-uvm-tools
134   - hostPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
135     containerPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
136   - hostPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
137     containerPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
138 - role: worker
139   extraMounts:
140   - hostPath: /dev/nvidia0
141     containerPath: /dev/nvidia0
142   - hostPath: /dev/nvidiactl
143     containerPath: /dev/nvidiactl
144   - hostPath: /dev/nvidiamodeset
145     containerPath: /dev/nvidiamodeset
146   - hostPath: /dev/nvidia-uvm
147     containerPath: /dev/nvidia-uvm
148   - hostPath: /dev/nvidia-uvm-tools
149     containerPath: /dev/nvidia-uvm-tools
150   - hostPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
151     containerPath: /usr/lib/x86_64-linux-gnu/libnvidia-ml.so.1
152   - hostPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1
153     containerPath: /usr/lib/x86_64-linux-gnu/libcuda.so.1

```

20.7 16.8: Script mega_setup.sh

Este script automatiza completamente el proceso de instalación y configuración del clúster Kind con soporte para GPUs NVIDIA. Incluye verificaciones de prerequisites, instalación de dependencias, configuración de Docker, creación del clúster y pruebas exhaustivas. Debe ejecutarse con privilegios de sudo.

```

1  #!/bin/bash
2
3  # Mega setup script for Kind cluster with GPU support
4  # Handles prerequisites, installations, configurations, and testing
5  # Must be run with sudo: sudo ./scripts/mega_setup.sh
6
7  set -o pipefail # Exit on pipe failures
8
9  # Change to project root directory
10 cd "$(dirname "$0")/.." || { echo "ERROR: Failed to change to project root"; exit 1; }
11
12 # Log file
13 LOG_FILE="mega_setup_$(date +%Y%m%d_%H%M%S).log"
14 exec > >(tee -a "$LOG_FILE") 2>&1
15
16 # Check if running as root
17 if [ "$EUID" -ne 0 ]; then
18     echo "ERROR: This script must be run with sudo privileges."
19     echo "Usage: sudo ./scripts/mega_setup.sh"
20     exit 1
21 fi
22
23 # Function to install package if missing
24 install_if_missing() {
25     local package=$1
26     if ! dpkg -l | grep -q "^ii\s$package"; then
27         echo "Installing $package..."
28         apt update && apt install -y "$package"
29         return $?
30     else
31         echo "$package already installed"
32         return 0
33     fi
34 }
35
36 # Colors for output
37 RED='\033[0;31m'
38 GREEN='\033[0;32m'
39 YELLOW='\033[1;33m'
40 BLUE='\033[0;34m'
41 NC='\033[0m' # No Color
42
43 # Global success flag
44 SUCCESS=true
45 TOTAL_STEPS=9
46 CURRENT_STEP=0
47
48 # Function to print step header
49 print_step() {

```

```

50     CURRENT_STEP=$((CURRENT_STEP + 1))
51     echo -e "${BLUE}Step_${CURRENT_STEP}/${TOTAL_STEPS}:_$_1${NC}"
52     echo "Remaining_steps:_${$((TOTAL_STEPS - CURRENT_STEP))}"
53 }
54
55 # Function to print colored output
56 print_status() {
57     local color=$1
58     local message=$2
59     echo -e "${color}${message}${NC}"
60 }
61
62 # Function to check command success
63 check_command() {
64     if [ $? -eq 0 ]; then
65         print_status $GREEN "OK:$_1"
66     else
67         print_status $RED "ERROR:$_1_failed"
68         SUCCESS=false
69     fi
70 }
71
72 # Initial checks
73 print_status $YELLOW "Realizando_verificaciones_iniciales..."
74
75 # Check internet connection
76 if ! ping -c 1 google.com &> /dev/null; then
77     print_status $RED "ERROR:_No_hay_conexion_a_internet._Requerida_
78     para_descargas."
79     exit 1
80 fi
81 print_status $GREEN "OK:_Conexion_a_internet_OK"
82
83 # Check NVIDIA drivers
84 if ! nvidia-smi &> /dev/null; then
85     print_status $RED "ERROR:_NVIDIA_drivers_no_detectados._Instala_los
86     _drivers_NVIDIA_primeramente."
87     exit 1
88 fi
89 print_status $GREEN "OK:_Drivers_NVIDIA_OK"
90
91 # Install jq if needed for JSON merging
92 install_if_missing jq
93 check_command "Instalacion_de_jq_(para_fusion_JSON)"
94
95 # Paso 1: Verificar e instalar Docker
96 # Docker es necesario para ejecutar contenedores, incluyendo los nodos
97 # de Kind
98 print_step "Verificar_e_instalar_Docker"
99 print_status $YELLOW "Comprobando_si_Docker_esta_instalado..."
100 if ! command -v docker &> /dev/null; then
101     print_status $YELLOW "Instalando_Docker_(actualiza_repositorios_y_
102     instala_docker.io)..."
103     apt update && apt install -y docker.io
104     check_command "Instalacion_de_Docker"
105     print_status $YELLOW "Iniciando_y_habilitando_el_servicio_Docker..."
106     "
107     systemctl start docker

```



```

103     systemctl enable docker
104     check_command "Inicio del servicio Docker"
105 else
106     print_status $GREEN "OK: Docker ya esta instalado"
107 fi
108
109 # Paso 2: Verificar e instalar nvidia-container-toolkit
110 # Este toolkit permite que Docker use el runtime de NVIDIA para
111 # contenedores GPU
112 print_step "Verificar e instalar nvidia-container-toolkit"
113 print_status $YELLOW "Comprobando si nvidia-container-toolkit esta
114     instalado..."
115 if ! dpkg -l | grep -q nvidia-container-toolkit; then
116     print_status $YELLOW "Instalando nvidia-container-toolkit..."
117     apt install -y nvidia-container-toolkit
118     check_command "Instalacion de nvidia-container-toolkit"
119 else
120     print_status $GREEN "OK: nvidia-container-toolkit ya esta instalado"
121 fi
122
123 # Paso 3: Verificar e instalar docker-compose
124 # Docker Compose facilita la gestion de multiples contenedores
125 print_step "Verificar e instalar docker-compose"
126 print_status $YELLOW "Comprobando si docker-compose esta disponible..."
127 if ! command -v docker-compose && ! docker compose version
128     && ! docker compose version > /dev/null; then
129     print_status $YELLOW "Instalando docker-compose-plugin..."
130     apt install -y docker-compose-plugin
131     check_command "Instalacion de docker-compose"
132 else
133     print_status $GREEN "OK: docker-compose ya esta disponible"
134 fi
135
136 # Paso 4: Configurar daemon.json de Docker
137 # Configura el runtime de NVIDIA como predeterminado para usar GPUs en
138 # contenedores
139 print_step "Configurar daemon.json de Docker"
140 print_status $YELLOW "Configurando /etc/docker/daemon.json para soporte
141     GPU..."
142 DAEMON_FILE="/etc/docker/daemon.json"
143 NVIDIA_CONFIG='{
144     "default-runtime": "nvidia",
145     "runtimes": {
146         "nvidia": {
147             "path": "nvidia-container-runtime",
148             "runtimeArgs": []
149         }
150     }
151 }'
152
153 if [ -f "$DAEMON_FILE" ]; then
154     print_status $YELLOW "Fusionando configuracion con daemon.json
155     existente..."
156     # Usa jq para fusionar si esta disponible, sino Python
157     if command -v jq && ! jq > /dev/null; then
158         jq '.+'"$NVIDIA_CONFIG"' "$DAEMON_FILE" > /tmp/daemon.json &&
159         mv /tmp/daemon.json "$DAEMON_FILE"

```

```

153     else
154         python3 -c "
155 import json
156 with open('${DAEMON_FILE}', 'r') as f:
157     existing=json.load(f)
158     existing.update(json.loads(''$NVIDIA_CONFIG'''))
159     with open('/tmp/daemon.json', 'w') as f:
160         json.dump(existing, f, indent=2)
161     """ && mv /tmp/daemon.json "$DAEMON_FILE"
162     fi
163     check_command "Fusion de daemon.json"
164 else
165     print_status $YELLOW "Creando nuevo daemon.json..."
166     echo "$NVIDIA_CONFIG" | tee "$DAEMON_FILE" > /dev/null
167     check_command "Creacion de daemon.json"
168 fi
169
170 # Reiniciar Docker para aplicar cambios
171 print_status $YELLOW "Reiniciando Docker para aplicar configuracion..."
172 systemctl restart docker
173 check_command "Reinicio de Docker"
174
175 # Paso 5: Instalar Kind y kubectl
176 # Kind crea clusters K8s locales; kubectl es el cliente para
    interactuar con K8s
177 print_step "Instalar Kind y kubectl"
178 print_status $YELLOW "Comprobando si Kind esta instalado..."
179 if ! command -v kind &> /dev/null; then
180     print_status $YELLOW "Descargando e instalando Kind..."
181     curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.30.0/kind-linux-
        amd64
182     chmod +x ./kind
183     mv ./kind /usr/local/bin/kind
184     check_command "Instalacion de Kind"
185 else
186     print_status $GREEN "OK: Kind ya esta instalado"
187 fi
188
189 print_status $YELLOW "Comprobando si kubectl esta instalado..."
190 if ! command -v kubectl &> /dev/null; then
191     print_status $YELLOW "Descargando e instalando kubectl..."
192     curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/
        release/stable.txt)/bin/linux/amd64/kubectl"
193     chmod +x kubectl
194     mv kubectl /usr/local/bin/kubectl
195     check_command "Instalacion de kubectl"
196 else
197     print_status $GREEN "OK: kubectl ya esta instalado"
198 fi
199
200 print_status $YELLOW "Comprobando si kubectl está instalado..."
201 if ! command -v kubectl &> /dev/null; then
202     print_status $YELLOW "Descargando e instalando kubectl..."
203     curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/
        release/stable.txt)/bin/linux/amd64/kubectl"
204     chmod +x kubectl
205     mv kubectl /usr/local/bin/kubectl
206     check_command "Instalacion de kubectl"

```

```

207 else
208     print_status $GREEN "OK:▯kubectl▯ya▯está▯instalado"
209 fi
210
211 # Paso 6: Verificar/crear cluster Kind
212 # Crea el cluster Kubernetes si no existe, usando la configuracion con
    soporte GPU
213 print_step "Verificar/crear▯cluster▯Kind"
214 print_status $YELLOW "Comprobando▯si▯el▯cluster▯Kind▯existe..."
215 CLUSTER_EXISTS=false
216 if kind get clusters 2>/dev/null | grep -q "^kind$"; then
217     CLUSTER_EXISTS=true
218     print_status $GREEN "OK:▯El▯cluster▯Kind▯ya▯existe"
219 else
220     print_status $YELLOW "Creando▯cluster▯Kind▯con▯configuracion▯GPU..."
221     "
222     kind create cluster --config config/kind-config.yaml
223     check_command "Creacion▯del▯cluster"
224     # Install NVIDIA device plugin for GPU scheduling
225     print_status $YELLOW "Instalando▯NVIDIA▯device▯plugin▯para▯
        scheduling▯GPU..."
226     kubectl apply -f https://raw.githubusercontent.com/NVIDIA/k8s-
        device-plugin/v0.17.0/deployments/static/nvidia-device-plugin.
        yml
227     check_command "Instalacion▯del▯device▯plugin▯NVIDIA"
228 fi
229
230 # Paso 7: Asegurar montaje de GPU en workers
231 # Si el cluster existe, copia nvidia-smi a los nodos worker para
    pruebas
232 print_step "Asegurar▯montaje▯de▯GPU▯en▯workers"
233 if [ "$CLUSTER_EXISTS" = true ]; then
234     print_status $YELLOW "Copiando▯nvidia-smi▯a▯los▯nodos▯worker▯para▯
        pruebas▯GPU..."
235     for worker in kind-worker kind-worker2 kind-worker3 kind-worker4;
236     do
237         if docker ps | grep -q "$worker"; then
238             docker cp /usr/bin/nvidia-smi "$worker":/usr/bin/nvidia-smi
239             2>/dev/null || true
240         fi
241     done
242     print_status $GREEN "OK:▯Montaje▯de▯GPU▯asegurado"
243 else
244     print_status $GREEN "OK:▯Saltado▯(cluster▯nuevo▯creado▯con▯montajes
        ▯incluidos)"
245 fi
246 done
247 print_status $GREEN "OK:▯Montaje▯de▯GPU▯asegurado"
248 else
249     print_status $GREEN "OK:▯Saltado▯(clúster▯nuevo▯creado▯con▯montajes
        ▯incluidos)"
250 fi
251
252 # Paso 8: Probar funcionalidad del cluster
253 # Verifica que los nodos esten listos, tengan etiquetas GPU, device
    plugin y que nvidia-smi funcione
254 print_step "Probar▯funcionalidad▯del▯cluster"
255 print_status $YELLOW "Verificando▯nodos▯del▯cluster..."

```

```

253
254 kubectl get nodes > /dev/null 2>&1
255 check_command "Verificacion_de_nodos_del_cluster"
256
257 print_status $YELLOW "Verificando_etiquetas_GPU_en_nodos..."
258 kubectl get nodes --show-labels | grep -q "nvidia.com/gpu.present=true"
259 check_command "Verificacion_de_etiquetas_GPU"
260
261 print_status $YELLOW "Verificando_NVIDIA_device_plugin..."
262 kubectl get pods -n kube-system | grep -q nvidia-device-plugin
263 check_command "Verificacion_del_device_plugin_NVIDIA"
264
265 print_status $YELLOW "Probando_acceso_GPU_en_nodos_worker..."
266 GPU_TEST_PASSED=true
267 for worker in kind-worker kind-worker2 kind-worker3 kind-worker4; do
268     if docker ps | grep -q "$worker"; then
269         if ! docker exec "$worker" nvidia-smi > /dev/null 2>&1; then
270             GPU_TEST_PASSED=false
271             break
272         fi
273     fi
274 done
275 if [ "$GPU_TEST_PASSED" = true ]; then
276     print_status $GREEN "OK:_Prueba_de_GPU_exitosa"
277 else
278     print_status $RED "ERROR:_Prueba_de_GPU_fallida"
279     SUCCESS=false
280 fi
281 fi
282 done
283 if [ "$GPU_TEST_PASSED" = true ]; then
284     print_status $GREEN "OK:_Prueba_de_GPU_exitosa"
285 else
286     print_status $RED "ERROR:_Prueba_de_GPU_fallida"
287     SUCCESS=false
288 fi
289
290 # Paso 9: Reportar estado final y resumen
291 # Indica si todo funciona correctamente o si hubo fallos, y resume lo
    realizado
292 print_step "Reportar_estado_final_y_resumen"
293 if [ "$SUCCESS" = true ]; then
294     print_status $GREEN "SUCCESS:_Todas_las_verificaciones_pasaron!_El_
        cluster_Kind_con_soporte_GPU_esta_listo."
295     echo ""
296     echo "Resumen_de_lo_realizado:"
297     echo "___Docker_instalado/configurado_con_runtime_NVIDIA"
298     echo "___nvidia-container-toolkit_instalado"
299     echo "___docker-compose_instalado"
300     echo "___Kind_y_kubectl_instalados"
301     echo "___Cluster_Kind_creado_con_1_control-plane_y_4_workers"
302     echo "___GPUs_montadas_en_todos_los_nodos"
303     echo "___NVIDIA_device_plugin_instalado"
304     echo "___Todas_las_pruebas_pasaron"
305     echo ""
306     echo "Log_guardado_en:_$LOG_FILE"
307 else
308     print_status $RED "ERROR:_Algunos_pasos_fallaron._Revisa_la_salida_"

```

```
309     anterior_para_detalle."
310     echo "Log guardado en: $LOG_FILE"
311 fi
```

20.8 16.7: Archivo kubeconfig.yaml

```

1  apiVersion: v1
2  clusters:
3  - cluster:
4      certificate-authority-data:
5          LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ01JUzhkMVVS
6          server: https://127.0.0.1:35123
7      name: kind-kind
8  contexts:
9  - context:
10      cluster: kind-kind
11      user: kind-kind
12      name: kind-kind
13  current-context: kind-kind
14  kind: Config
15  users:
16  - name: kind-kind
17      user:
18      client-certificate-data:
19          LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSB0tLS0tCk1JSURLVENDQWwHZ0F3SUJBZ01JTGNkQ1VmeVdI
20      client-key-data:
21          LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQpNSU1FcEFJQkFBS0NBUEVBemJvMG4xSzJQ
22          ...

```

20.9 16.9: Script cluster_manager.py

Este script en Python proporciona una interfaz programática para gestionar clústeres Kind. Permite crear, eliminar, verificar el estado del clúster, instalar plugins GPU y ArgoCD de manera automatizada.

El contenido completo del script se encuentra en `scripts/cluster_manager.py`.

20.10 16.10: Script gpu_monitor.py

Este script en Python monitorea el uso de recursos GPU en el clúster Kubernetes. Proporciona estadísticas en tiempo real sobre nodos GPU, pods que utilizan GPU y utilización general.

El contenido completo del script se encuentra en `scripts/gpu_monitor.py`.

20.11 16.11: Archivo argocd-app.yaml

Este archivo YAML define una aplicación de ejemplo para ArgoCD, permitiendo el despliegue continuo de aplicaciones desde un repositorio Git.

```

1  apiVersion: argoproj.io/v1alpha1
2  kind: Application
3  metadata:

```

```
4   name: my-app
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: https://github.com/my-org/my-app
10    targetRevision: HEAD
11    path: .
12 destination:
13   server: https://kubernetes.default.svc
14   namespace: default
```

21 Bibliografía

Referencias

- [1] Kind. (2025). Documentation. <https://kind.sigs.k8s.io/docs/>. Accedido el 13 de noviembre de 2025.
- [2] Kind. (2025). Quick Start. <https://kind.sigs.k8s.io/docs/user/quick-start/>. Accedido el 13 de noviembre de 2025.
- [3] Kubernetes Documentation. <https://kubernetes.io/docs/>. Accedido el 13 de noviembre de 2025.
- [4] NVIDIA GPU Device Plugin for Kubernetes. <https://github.com/NVIDIA/k8s-device-plugin>. Accedido el 13 de noviembre de 2025.
- [5] Docker Documentation. <https://docs.docker.com/>. Accedido el 13 de noviembre de 2025.
- [6] NVIDIA Container Toolkit. <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/overview.html>. Accedido el 13 de noviembre de 2025.
- [7] ArgoCD Documentation. <https://argo-cd.readthedocs.io/>. Accedido el 13 de noviembre de 2025.
- [8] k9s Terminal UI for Kubernetes. <https://k9scli.io/>. Accedido el 13 de noviembre de 2025.
- [9] Lens - The Kubernetes IDE. <https://k8slens.dev/>. Accedido el 13 de noviembre de 2025.
- [10] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1), 70-93.
- [11] Kind: Kubernetes in Docker. (2020). GitHub Repository. <https://github.com/kubernetes-sigs/kind>. Accedido el 13 de noviembre de 2025.
- [12] NVIDIA. (2025). GPU Operator for Kubernetes. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/overview.html>. Accedido el 13 de noviembre de 2025.
- [13] Docker Swarm vs Kubernetes. (2023). Comparison Article. <https://www.docker.com/blog/docker-swarm-vs-kubernetes/>. Accedido el 13 de noviembre de 2025.
- [14] ArgoCD: Declarative GitOps CD for Kubernetes. (2020). ArgoCD Blog. <https://argo-cd.readthedocs.io/en/stable/>. Accedido el 13 de noviembre de 2025.
- [15] k9s: Kubernetes CLI To Manage Your Clusters In Style. (2019). GitHub. <https://github.com/derailed/k9s>. Accedido el 13 de noviembre de 2025.
- [16] Kubernetes Security Best Practices. (2025). Kubernetes Documentation. <https://kubernetes.io/docs/concepts/security/>. Accedido el 13 de noviembre de 2025.

-
- [17] CUDA Toolkit Documentation. (2025). NVIDIA. <https://docs.nvidia.com/cuda/>. Accedido el 13 de noviembre de 2025.
 - [18] etcd Documentation. (2025). <https://etcd.io/docs/>. Accedido el 13 de noviembre de 2025.
 - [19] containerd: An industry-standard container runtime. (2025). <https://containerd.io/>. Accedido el 13 de noviembre de 2025.
 - [20] Kubelet Component. (2025). Kubernetes. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/>. Accedido el 13 de noviembre de 2025.
 - [21] Docker Compose Documentation. (2025). <https://docs.docker.com/compose/>. Accedido el 13 de noviembre de 2025.
 - [22] NVIDIA Driver Installation. (2025). <https://docs.nvidia.com/datacenter/tesla/tesla-installation-notes/>. Accedido el 13 de noviembre de 2025.
 - [23] Kubernetes Networking. (2025). <https://kubernetes.io/docs/concepts/cluster-administration/networking/>. Accedido el 13 de noviembre de 2025.
 - [24] ArgoCD Getting Started. (2025). https://argo-cd.readthedocs.io/en/stable/getting_started/. Accedido el 13 de noviembre de 2025.
 - [25] k9s User Guide. (2025). <https://k9scli.io/topics/>. Accedido el 13 de noviembre de 2025.
 - [26] Lens Documentation. (2025). <https://docs.k8slens.dev/>. Accedido el 13 de noviembre de 2025.
 - [27] Pods. (2025). Kubernetes Concepts. <https://kubernetes.io/docs/concepts/workloads/pods/>. Accedido el 13 de noviembre de 2025.